

GrowAI LLM Engineering — Runbook

How to run Assignments 7, 10 and 11 · generated 9 September 2026

Assignment 7	GraphRAG Knowledge Explorer with Neo4j	verified 15/15
Assignment 10	Fine-Tune & Deploy a Domain-Specific LLM	partly verified
Assignment 11	Production LLM API with Evals & Observability	never executed

Shared prerequisites

```
cd ~/PycharmProjects/PythonProject/GrowAiAssignments
source ../.venv/bin/activate
ollama serve # separate terminal, leave running
```

Model	Used by
llama3.2	Assignment 7 (extraction + answers), Assignment 11 (generation)
omic-embed-text	Assignment 11 (embeddings, keeps torch out of the image)
qwen2.5:1.5b	Assignment 10 (baseline for the comparison)

Memory is the recurring constraint. This is an 8 GB M1. Ollama with a loaded model, plus Neo4j, plus two Docker containers will not coexist. Run one assignment at a time and stop the others first. Exit code 137 on a container means it was OOM-killed — that has already happened to Neo4j here.

Assignment 7 — GraphRAG with Neo4j verified 15/15

Step 1 — Start Neo4j

```
docker start neo4j          # NOT docker run – the container already exists
```

Wait ~10 s, then confirm (expect 200):

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:7474
```

Credentials `neo4j` / `password123` already match cell 1 — no edits needed.

`docker run` only works once. A second attempt fails with `Conflict`. The container name `"/neo4j"` is already in use. Use `docker start`.

Step 2 — Run the notebook

```
jupyter notebook Assignment7_graphrag.ipynb
```

Run every cell top to bottom. Cell 8 begins with `MATCH (n:Entity) DETACH DELETE n`, so the graph is wiped and rebuilt each run — running cells out of order is what produces a half-populated graph.

Checkpoints

Cell	Expected output
3	Connected to Neo4j at bolt://localhost:7687
7	Chunks that produced no triples: 0 / 31 — if not 0, paragraphs were dropped
8	Reversed triples corrected: ~4, Unusable triples dropped: ~1
11	Nodes: ~68, Relationships: ~72
19	GraphRAG correct on all five multi-hop questions

Step 3 — Inspect the graph

Open `http://localhost:7474`, log in, then:

```
MATCH (n)-[r]->(m) RETURN n, r, m
```

Timing

Cell 7 (31 paragraphs through llama3.2) takes ~4 minutes; section 15 makes ten generations, ~3 minutes. Budget **8–10 minutes** total.

Status. GraphRAG answered 15/15 (3 trials × 5 questions) after the directed-path-chain fix, against 9/15 before — questions 1 and 2 went from 0/3 to 3/3. That measurement exercised the notebook's own cells 1 and 13–17 but skipped the extraction cells, so a full top-to-bottom run is still the real end-to-end confirmation.

Assignment 10 — Fine-Tune & Deploy partly verified

Four stages across two machines. The dataset is built and the model is compared locally through Ollama; the fine-tune and the GGUF export run on **Google Colab with a GPU**, which is what the brief specifies.

Stage	Where	File
1. Generate dataset	local	Assignment10_dataset.py
1b. Validate	local	Assignment10_validate.py
2. LoRA fine-tune	Colab GPU	Assignment10_finetune.ipynb
3. Merge + export GGUF	Colab GPU	same notebook
4. Register + compare	local	Assignment10_Modelfile , Assignment10_compare.py
All of these live in Assignment10_finetuned/		

Step 1 — Generate the dataset done

```
cd Assignment10_finetuned
python Assignment10_dataset.py          # -> fastapi_dataset.json
```

Currently 74 pairs across 16/16 topics. Safe to re-run: it resumes and refills only short topics. A local Ollama model is the teacher here, so `ollama serve` must be running.

Step 1b — Validate done

```
python Assignment10_validate.py
```

Last run kept 74/74 with 0 rejected. Always re-run after regenerating — an earlier batch had 13 of 51 pairs containing invented APIs such as `from fastapi import CORS`. Training on those teaches the model to emit them confidently, which is worse than not fine-tuning at all.

Step 2 — Fine-tune on Colab start here

1. Upload `Assignment10_finetuned/Assignment10_finetune.ipynb` to `colab.research.google.com`.
2. **Runtime** → **Change runtime type** → **T4 GPU**. Without a GPU the notebook will fail at the first Unsloth call.
3. Upload `fastapi_dataset.json` using the Files panel on the left, or run `from google.colab import files; files.upload()`.
4. Run all cells top to bottom.

What the notebook does

Section	Step
1	Installs Unsloth and pinned dependencies
2	Loads <code>unsloth/Qwen2.5-1.5B-bnb-4bit</code> in 4-bit (QLoRA)

3	Records base-model answers to the 5 evaluation questions <i>before</i> the adapter is attached
4	Attaches LoRA r=16 on attention projections, prints trainable % (must be < 5%)
5	Loads <code>fastapi_dataset.json</code> and formats it into the Alpaca prompt
6	Trains 60 steps at lr 2e-4, logging loss every 10
7	Prints runtime and the loss curve
8	Prints base vs fine-tuned answers side by side
9	Saves the LoRA adapter (tens of MB — the base model is unchanged)
10	Merges the adapter and exports GGUF at Q4_K_M
11	Downloads the GGUF file

Capture the baseline before training, not after. Cell 3 exists so the comparison is against recorded output rather than a remembered impression — that is how people convince themselves a fine-tune worked when it did not.

Step 3 — Download the GGUF

```
from google.colab import files
files.download('fastapi_model_gguf/unsloth.Q4_K_M.gguf')
```

Section 11 of the notebook does this. Roughly 1 GB for a 1.5B model at Q4_K_M, so the browser download is slow. Mounting Google Drive and copying there is usually faster. Colab sessions also expire — download before the runtime disconnects or the file is lost.

Step 4 — Register locally and compare

```
cd Assignment10_finetuned
# put the downloaded .gguf here, then point the FROM line at it
ollama create fastapi-expert -f Assignment10_Modelfile
python Assignment10_compare.py
```

`qwen2.5:1.5b` is already pulled as the baseline. The comparison script degrades gracefully when `fastapi-expert` is missing, so it can be run early to see base-model answers alone.

The Modelfile repeats the Alpaca prompt template on purpose. The model was trained in that format; a mismatch between training and inference is the usual reason a fine-tune appears to do nothing.

Status. Stages 1 and 1b are complete and verified here — 74 clean pairs, 0 rejected. Stages 2 and 3 have not been run on Colab from this machine, so treat the notebook as untested end to end. Expect the honest limitation of the result too: ~74 synthetic pairs from a small teacher model shift *style*, not capability. Fine-tuning teaches behaviour; RAG supplies facts.

Assignment 11 — Production LLM API never executed

Component	Where	Port
FastAPI service	Docker container	8080
ChromaDB vector store	Docker container	8000
Ollama	host machine, not Docker	11434

Step 1 — Models

```
ollama pull llama3.2
ollama pull nomic-embed-text
```

Step 2 — Start the stack

```
cd ~/PycharmProjects/PythonProject/GrowAiAssignments/Assignment11_production_api
docker compose up --build
```

First run pulls `chromadb/chroma:0.6.3` and builds the API image — several minutes.

Step 3 — Health

```
curl -s http://localhost:8080/health | python3 -m json.tool
```

- **chroma false** — still starting, wait a few seconds.
- **ollama false** — `host.docker.internal` not resolving from the container.

Step 4 — Endpoints

```
curl -N -X POST http://localhost:8080/chat \
  -H "Content-Type: application/json" \
  -d '{"query": "What is deep learning?", "stream": true}'

curl -X POST http://localhost:8080/chat \
  -H "Content-Type: application/json" \
  -d '{"query": "What is deep learning?", "stream": false}' | python3 -m json.tool
```

Run the non-streaming call twice. The first is a cache miss; the second returns `"cached": true` with much lower latency — that is the cache demonstration the brief asks for.

Steps 5–7 — Client, observability, evals

```
python client.py                # streaming + cache speedup side by side
curl -s http://localhost:8080/metrics | python3 -m json.tool
tail -5 logs/requests.jsonl

pip install -r requirements-dev.txt # deepeval
python evals.py                    # run from the HOST, stack up. Several minutes.
```

Step 8 — Teardown

```
docker compose down
```

```
# -v also drops the Chroma volume
```

Status. This stack has never run end to end. Docker was not installed when it was built, so it is verified only by syntax checks, a YAML parse, and a smoke test of `main.py` against a stubbed backend — which did find and fix two real bugs. The Docker layer itself (image build, `host.docker.internal`, container networking) is unproven. Expect first-run friction.

Troubleshooting — all three

Symptom	Cause / fix
Container exits with 137	OOM-killed. Stop other containers, close apps. The recurring failure on 8 GB.
<code>Conflict ... name "/neo4j" is already in use</code>	Use <code>docker start neo4j</code> , not <code>docker run</code> .
A7: empty graph, GraphRAG matches vector RAG	Extraction failed or cells ran out of order. Check <code>Chunks</code> that produced <code>no triples</code> .
A10: Unsloth fails on the first call	No GPU attached. Runtime → Change runtime type → T4 GPU.
A10: session disconnects mid-training	Colab free tier idles out. Re-run from the top; download the GGUF as soon as it exists.
A10: fine-tuned model answers like the base one	Prompt-template mismatch. The Modelfile must use the Alpaca format the model was trained on.
A11: <code>port</code> is already allocated	<code>lsof -nP -iTCP:8080 -sTCP:LISTEN</code> to find the holder.
A11: answers are always "Not in context."	Index not built. Look for <code>Vector index ready: N chunks</code> in the API logs.
Ollama unreachable	<code>curl http://localhost:11434/api/tags</code> . Restart with <code>ollama serve</code> .